

Automation with Selenium

Assignment 4: JavaScript Alerts and Confirms

1. Problem Statement: -

Trigger a JavaScript Alert, a Confirm Box, and a Prompt Box on a practice page. Accept the first alert, dismiss the confirm box, and use driver.switch_to.alert.send_keys() to input text into the prompt before submitting.

2. Objective: -

To handle browser-native JavaScript popup windows which cannot be inspected via standard DOM locators.

3. Tools, Software, and Concepts Used: -

Python 3.x, Selenium WebDriver, Alert Interface.

4. Source Code / Implementation: -

```
from selenium import webdriver
from selenium.webdriver.common.by import By
import time
driver = webdriver.Chrome()
driver.get("https://the-internet.herokuapp.com/javascript_alerts")
time.sleep(2)
driver.find_element(By.XPATH, "//button[text()='Click for JS Alert']").click()
time.sleep(5)
alert = driver.switch_to.alert
alert.accept()
time.sleep(2)
driver.find_element(By.XPATH, "//button[text()='Click for JS Confirm']").click()
time.sleep(5)
confirm = driver.switch_to.alert
confirm.dismiss()
time.sleep(2)
driver.find_element(By.XPATH, "//button[text()='Click for JS Prompt']").click()
time.sleep(5)
prompt = driver.switch_to.alert
prompt.send_keys("Selenium Test")
prompt.accept()
time.sleep(2)
print("JavaScript alerts handled successfully.")
```

```
driver.quit()
```

5. Output & Screenshots: -

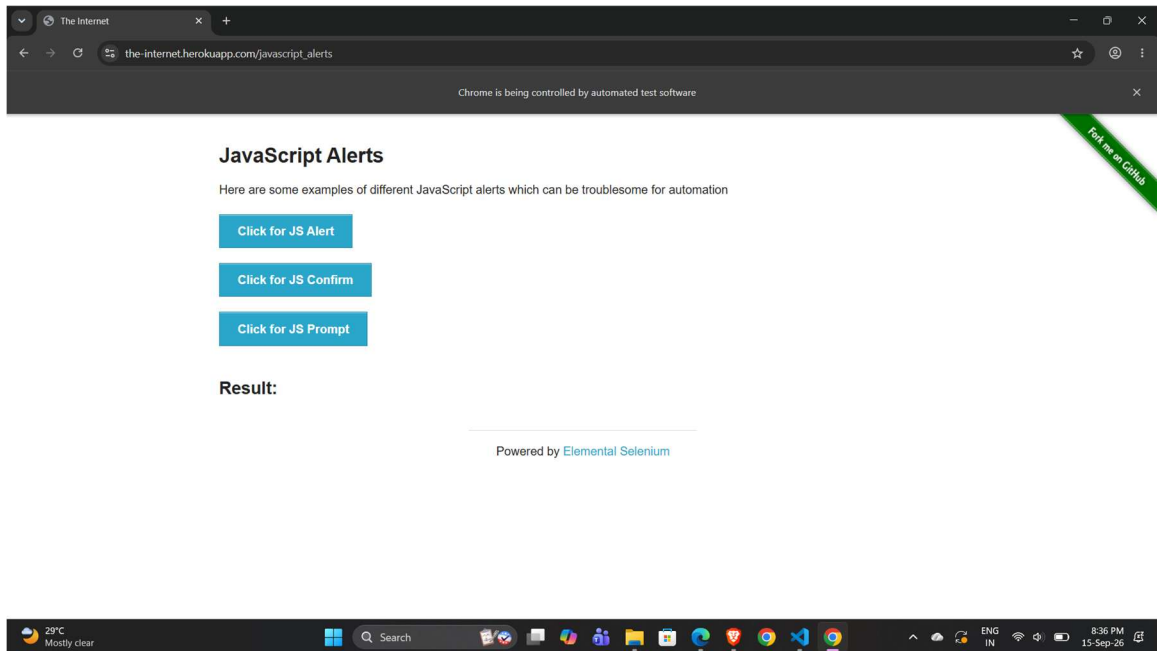


Fig.1: Alert page is visible

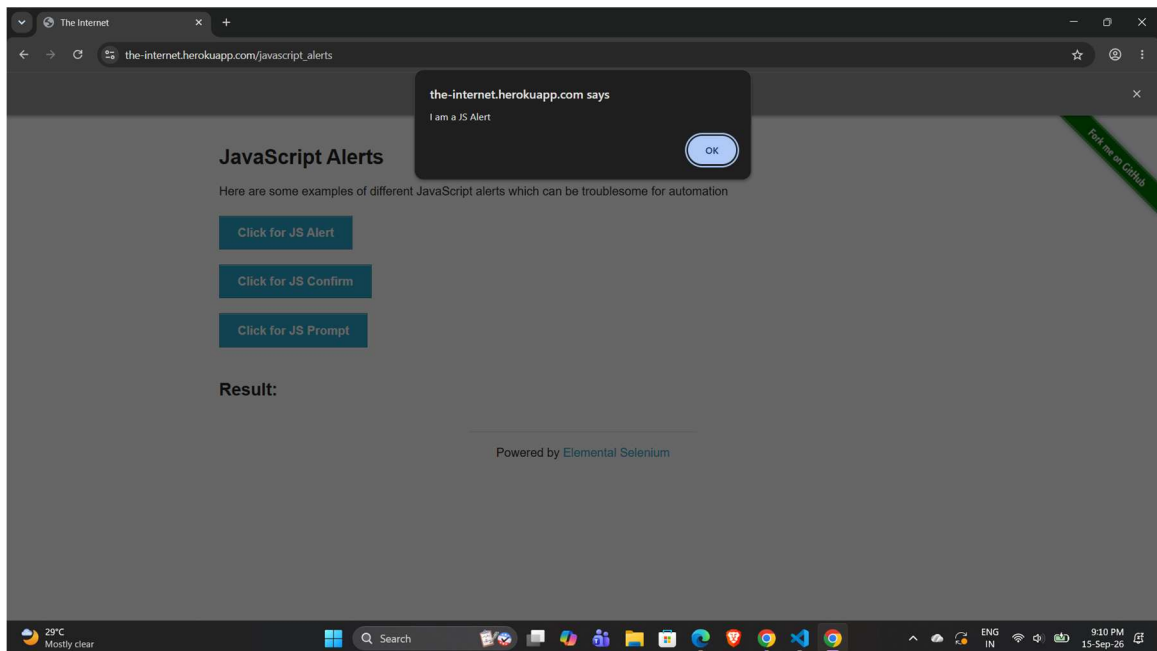


Fig.2: JS alert message is visible

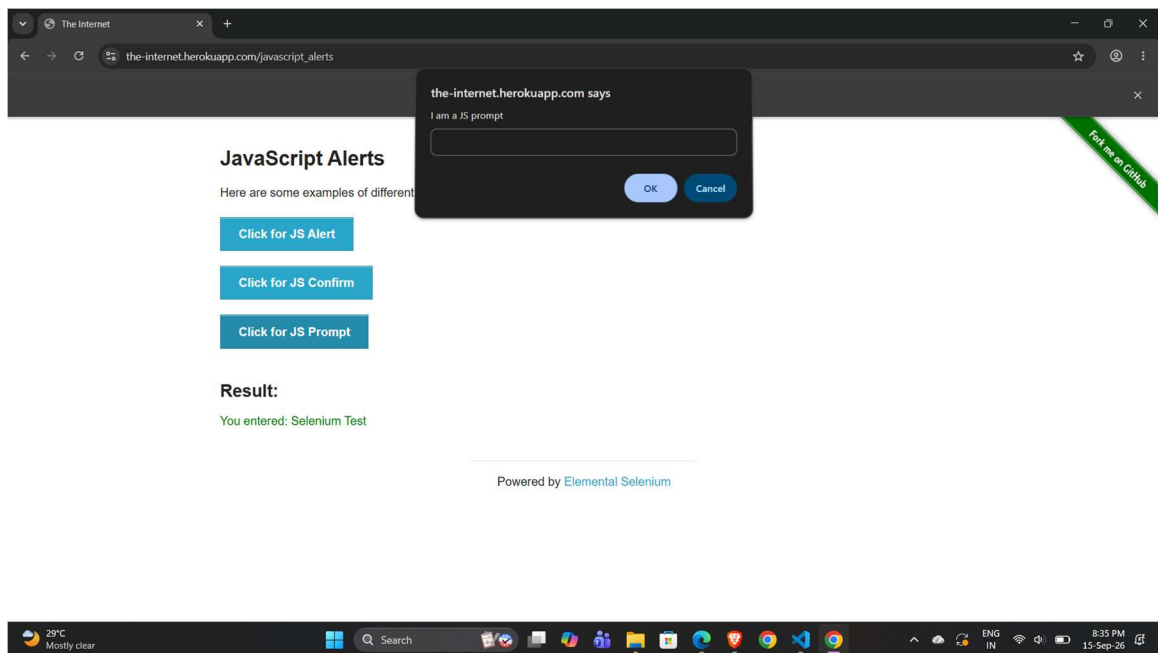


Fig.3: JS prompt is visible

6. Result, Observation, and Conclusion: -

- Result: Successfully interacted with standard Alert, Confirm, and Prompt dialogs.
- Observation: Browser-native dialogs halt execution; the context must be switched via `switch_to.alert` to interact with them.
- Conclusion: WebDriver's built-in alert handling provides a straightforward mechanism to bypass non-DOM blocking dialogs.